



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algoritmusok és Adatszerkezetek I.

Pusztai Gábor

Algoritmusok és Alkalmazásaik Tanszék
Informatikai Kar
ELTE - Eötvös Loránd Tudományegyetem, Budapest

2026. április 13.

- 1 Bináris keresőfa
 - Bináris keresőfa építése
 - Fa rekonstrukciója

- 2 Műveletek
 - Keresés
 - Beszúrás
 - Törlés
 - "Rákövetkező"

Bináris keresőfa

Bináris keresőfa - Binary search tree (BST) definíciója

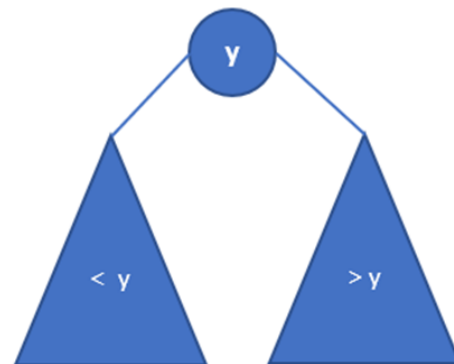
Egy bináris fát *keresőfának* nevezünk, ha minden belső csúcsára és annak y kulcsára igazak az alábbi követelmények:

- A csúcs bal részfájában tetszőleges csúcs x kulcsára $x < y$.
- A csúcs jobb részfájában tetszőleges csúcs z kulcsára $z > y$.

Egy bináris fát *rendezőfának* nevezünk, ha minden belső csúcsára és annak y kulcsára igazak az alábbi követelmények:

- A csúcs bal részfájában tetszőleges csúcs x kulcsára $x \leq y$.
- A csúcs jobb részfájában tetszőleges csúcs z kulcsára $z \geq y$.

A fa egy tetszőleges belső pontjára igaz az alábbi feltétel.



Mi a hiba a következő definícióban?

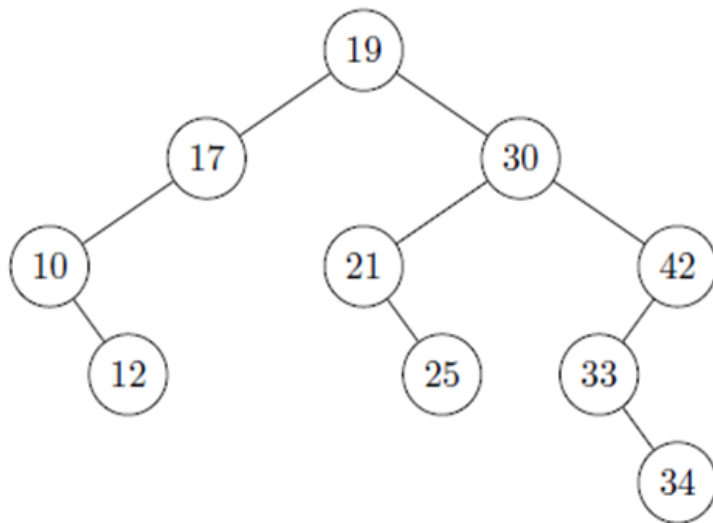
- „Minden belső csúcs bal gyerekének kulcsa kisebb, jobb gyerekének kulcsa nagyobb.”
- **Hiba:** Nem elég a csúcs gyerekeire kimondani, a csúcs **összes** bal, illetve jobb részfájában lévő kulcsra teljesülnie kell!

Feladat: Bináris keresőfa építése

- A kulcsok egymás utáni beszúrásával építsünk bináris keresőfát a következő elemekből:
- 19, 17, 30, 10, 21, 12, 42, 25, 33, 34

Megoldás: Bináris keresőfa építése

- A kulcsok egymás utáni beszúrásával építsünk bináris keresőfát a következő elemekből:
- 19, 17, 30, 10, 21, 12, 42, 25, 33, 34



- A preorder illetve postorder bejárásból rekonstruálhatjuk a bináris keresőfa alakját.

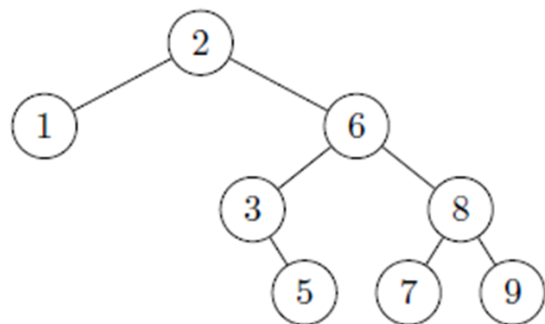
Preorder: 2, 1, 6, 3, 5, 8, 7, 9

Postorder: 2, 4, 3, 9, 8, 10, 7, 5

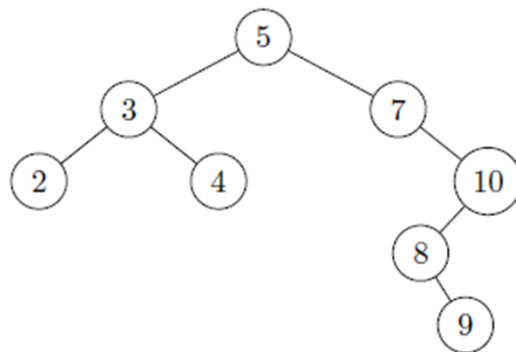
Megoldás: Fa rekonstrukciója

- A preorder illetve postorder bejárásból rekonstruálhatjuk a bináris keresőfa alakját.

Preorder: 2, 1, 6, 3, 5, 8, 7, 9



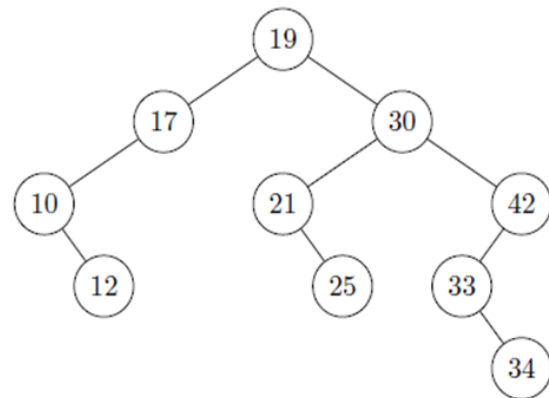
Postorder: 2, 4, 3, 9, 8, 10, 7, 5



Műveletek

Kérdések a felépített fához

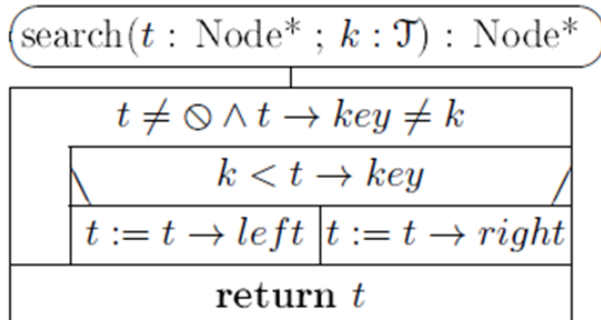
- Keressük meg a 25-ös kulcsot!
- Keressük meg a 23-as kulcsot!
- Milyen kulcsok illenének a 21-es bal részfájába? (20)
- Milyen kulcsok illenének a 12-es jobb részfájába? (13..16)
- Melyik bejárás adja a kulcsok nagyság szerinti növekvő felsorolását?
(Inorder)



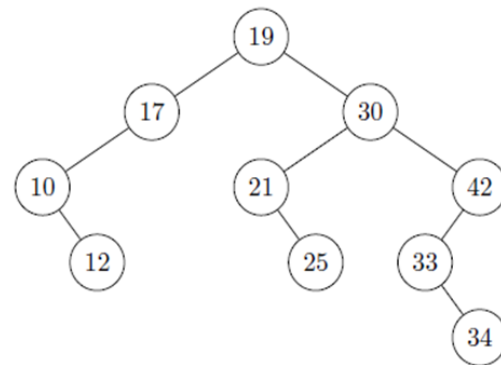
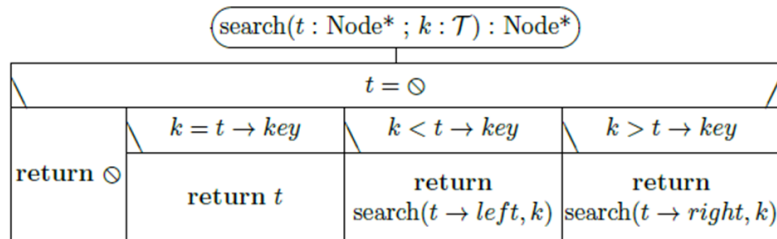
- A műveletekhez készíthetünk rekurzív, illetve iteratív változatot.
- **Műveletek:**
 - `search(t, k)`: k kulcsú elem keresése.
 - `insert(t, k)`: k kulcsú elem beszúrása.
 - `min(t)`: minimális kulcsú csúcs címe.
 - `remMin(t, minp)`: minimális elem kivétele.
 - `del(t, k)`: k kulcsú elem törlése.
- Mindegyik művelet $O(h)$ költségű, ahol h a fa magassága.

Keresés művelet (Iteratív és Rekurzív)

Iteratív:

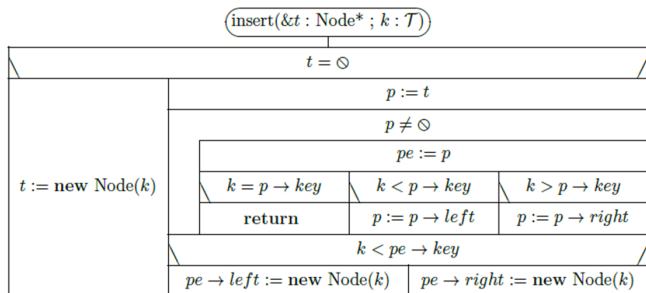


Rekurzív:

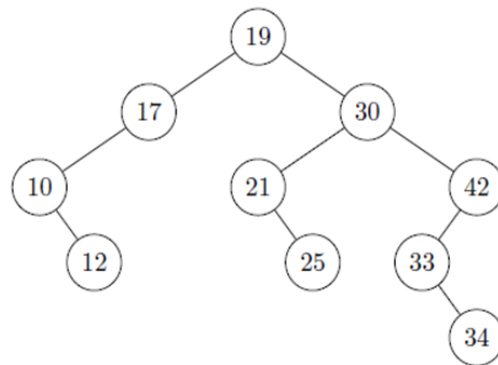
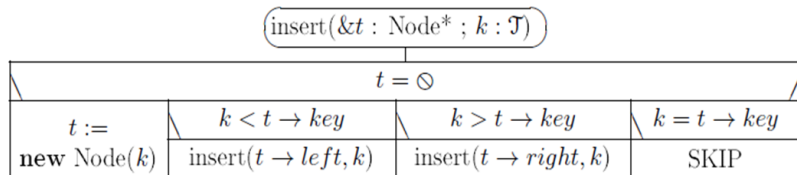


Beszúrás művelet (Iteratív és Rekurzív)

Iteratív:



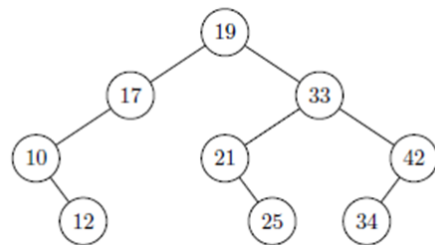
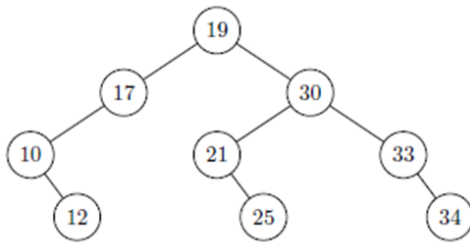
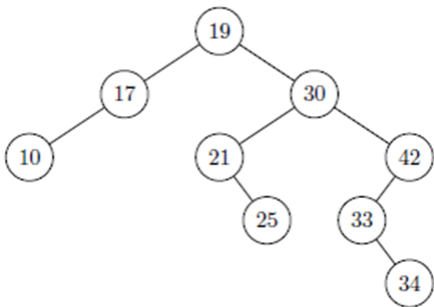
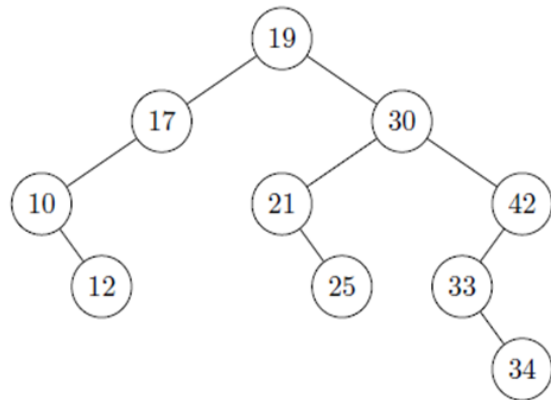
Rekurzív:



Törlés művelet esetei

- **Levél törlése:** pl. 12
- **Egy gyerekes belső pont törlése:** pl. 42
- **Két gyerekes belső pont törlése:** pl. 30 (jobb részfából a legkisebb elem kerül a helyére)

Eredeti fa:



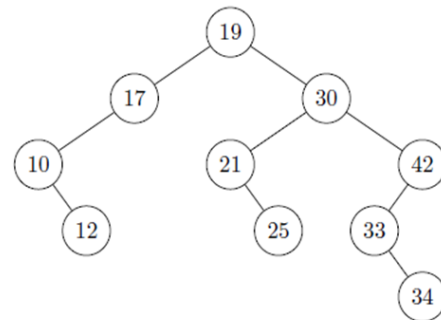
Törlés művelet

$\text{del}(\&t : \text{Node}^* ; k : \mathcal{T})$

$t \neq \ominus$			
$k < t \rightarrow \text{key}$	$k > t \rightarrow \text{key}$	$k = t \rightarrow \text{key}$	SKIP
$\text{del}(t \rightarrow \text{left}, k)$	$\text{del}(t \rightarrow \text{right}, k)$	$\text{delRoot}(t)$	

$\text{delRoot}(\&t : \text{Node}^*)$

$p := t$		
$t \rightarrow left = \ominus$	$t \rightarrow right = \ominus$	$t \rightarrow left \neq \ominus \wedge t \rightarrow right \neq \ominus$
$t := p \rightarrow right$	$t := p \rightarrow left$	$\text{remMin}(t \rightarrow right, q)$
		$q \rightarrow left := p \rightarrow left$
		$q \rightarrow right := p \rightarrow right$
		$t := q$
delete p		

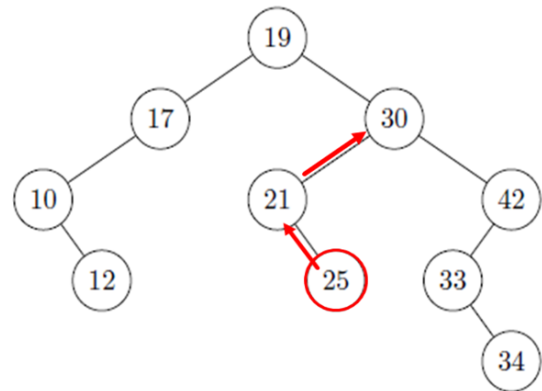
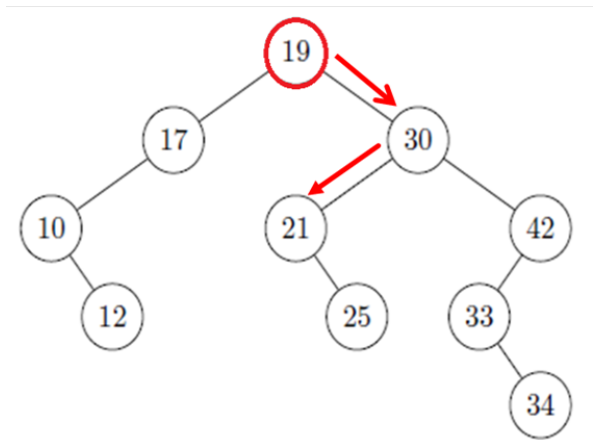


$\text{remMin}(\&t, \&\text{minp} : \text{Node}^*)$

$t \rightarrow \text{left} = \ominus$	
$\text{minp} := t$	$\text{remMin}(t \rightarrow \text{left}, \text{minp})$
$t := \text{minp} \rightarrow \text{right}$	
$\text{minp} \rightarrow \text{right} := \ominus$	

A „rákövetkező” művelet - $inorder_next(p)$

- Az aktuális elem inorder bejárás szerinti rákövetkezőjét keressük.
- A művelethez a szülő pointerre is szükség van!



- Ha van jobb részfája: annak minimuma.
- Ha nincs: az első olyan felmenő, akinek bal részfájában vagyunk.

A „rákövetkező” művelet - $inorder_next(p)$

$inorder_next(p : Node3^*) : Node3^*$

$q := p \rightarrow right$

$q \neq \emptyset$

$q \rightarrow left \neq \emptyset$

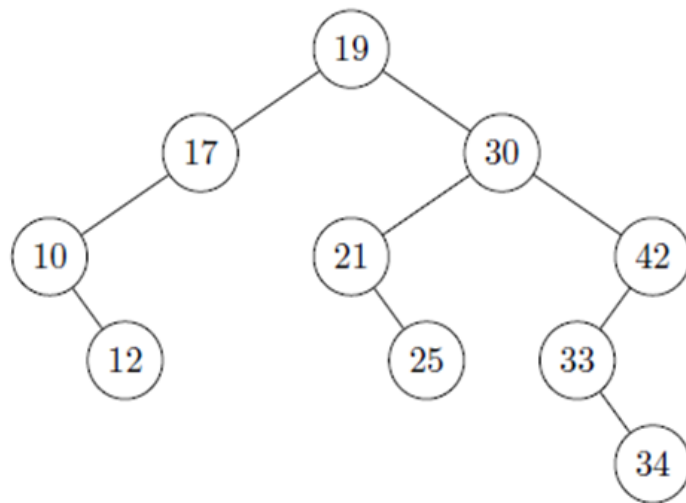
$q := p \rightarrow parent$

$q := q \rightarrow left$

$q \neq \emptyset \wedge q \rightarrow left \neq p$

$p := q ; q := q \rightarrow parent$

return q

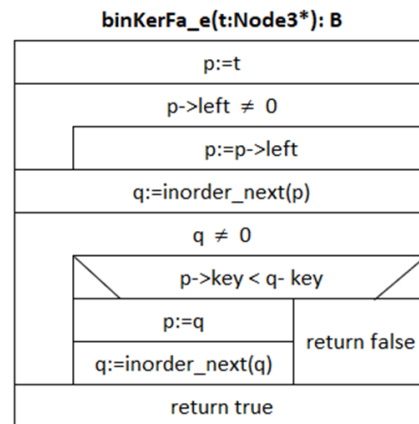


- **Feladat:** adott egy láncoltan ábrázolt bináris fa, ellenőrizzük, hogy bináris keresőfa-e. A fa csúcsai *Node3* típusúak, azaz szülő pointer is rendelkezésre áll. Feltehetjük, hogy a fa nem üres.

Bináris keresőfa tulajdonság ellenőrzése

- **Feladat:** adott egy láncoltan ábrázolt bináris fa, ellenőrizzük, hogy bináris keresőfa-e. A fa csúcsai *Node3* típusúak, azaz szülő pointer is rendelkezésre áll. Feltehetjük, hogy a fa nem üres.
- Egy iteratív algoritmust képzeljünk el, mely a most megismert *inorder_next(p)* algoritmus segítségével járja be a fát, és közben ellenőrzi, hogy szigorúan monoton növekvő-e a kulcsok sorrendje.

- **Ötlet:** Inorder bejárással ellenőrizzük a szigorúan monoton növekvő sorrendet.



Köszönöm a figyelmet!